# Test Plan — File Compression Tool (Huffman Coding, Python)

| Item | Detail |
|---|---|
| **Document** | Test Plan |
| **Product** | File Compression Tool — Huffman coding, Python (CLI + GUI) |
| **Version** | 1.0 |
| **Date** | 6 November 2026 |
| **Repository** | `KeerthanaKumaresh1/file-compression-tool` |
| **Team (GitHub usernames)** | KeerthanaKumaresh1 (Keerthana Kumaresh — repository owner), rishi2908, khushi309 |
| **Status** | Final for demo / viva, based on the executed test runs cited in §8 and in `Testing_Reports.md` |

> **Full names:** Only Keerthana Kumaresh is named in full in the evidence provided. The other two team members are identified here by GitHub username only (rishi2908, khushi309). Their full names were not in the evidence inspected — add them from the team list before submission if required.
>
> **How to read this plan:** Every result marked **Pass** in §8 is backed by a measured run cited in the same row. Nothing in this plan invents a test result, a PR number, or a coverage number. Items that could not be verified from the evidence provided are listed in §10 as not in the evidence, not invented.

## Revision History

| Version | Date | Author | Change |
|---|---|---|---|
| 0.1 | Sprint 1 | Team | Initial test cases UT_01–UT_10 drafted in `documents/Test_Cases.pdf`. Actual Results left as "To be filled after execution". |
| 1.0 | 6 Nov 2026 | Team | IEEE-format Test Plan. Sections 3, 4 and 5 completely filled. §5.1 dedicated to Security Validation. Traceability to SRS added (§9). All 10 test cases executed — results filled in §8 (all Pass), with the correction in UT_07 (`.huff` → `.hfc`) and the illustrative-only note in UT_10. Coverage and live system-validation evidence incorporated from `Testing_Reports.md`. |

## Contents


1. Introduction
2. Test Items
3. Features to be Tested

4. Features NOT to be Tested
5. Test Approach — 5.1 Security Validation, 5.2 Unit Testing, 5.3 Integration / System Testing, 5.4 Regression Testing
6. Test Environment
7. Entry / Exit Criteria
8. Test Cases UT_01–UT_10
9. Traceability Matrix
10. Risks & Limitations

---

# 1. Introduction

## 1.1 Purpose

This Test Plan defines what was tested, how it was tested, and what the measured results were for the File Compression Tool, a Python application that compresses files losslessly with Huffman coding and decompresses them back to byte-identical originals.

It is written to the IEEE test-plan format required by the mini-project checklist (`02TheCheckList.pdf`): Sections 3, 4 and 5 are completely filled, §5.1 is dedicated to security validation, traceability back to the SRS is established in §9, and it contains exactly 10 test cases (UT_01–UT_10) covering Functional Requirements (FRs) and Non-Functional Requirements (NFRs).

## 1.2 Scope

In scope:

- Automated unit tests for the `compressor` package (`tests/`, 81 tests).
- Integration / system validation by live CLI runs on real files (small and large).
- Line coverage and branch coverage of the `compressor` package.
- Security validation: invalid / truncated / corrupted input, CRC mismatch, wrong magic, overwrite protection, empty file, and error-message disclosure (§5.1).
- Traceability from each test case to SRS requirements, backlog items (BL1–BL20), and test files.

Out of scope: see §4.

## 1.3 References

| Ref. | Document |
|---|---|
| [1] | Software Requirements Specification (SRS), v2.0 — File Compression Tool, delivered with this Test Plan. FR-1–FR-14, NFR-1–NFR-8, security objectives SO-1–SO-4 and SR-1–SR-10 (wording kept from `documents/Security_Requirements.pdf`). |
| [2] | Software Architecture & Design (SAD), v1.0. |
| [3] | `documents/Test_Cases.pdf` — original UT_01–UT_10 (Actual Result "To be filled after execution" in the original; now filled in §8 of this plan). |

| Ref. | Document |
|---|---|
| [4] | `Testing_Reports.md` (companion to this plan, in the `docs` folder) — Unit Test Report, System Validation Report, Code Coverage Report, Branch Coverage Report. |
| [5] | Mini-project requirements: `01FAQs_MiniProjectDemo.pdf` and `02TheCheckList.pdf`. |
| [6] | GitHub Project `File_Compression_Tool_BLs` — Backlog, 20 items BL1–BL20, all Done, 99 story points (see §2.3). |
| [7] | Repository `KeerthanaKumaresh1/file-compression-tool` — source, `tests/`, `compressor/` package, `gui.py`. |

### 1.4 Definitions

- **Huffman coding:** a lossless compression method that gives shorter bit codes to more frequent symbols.
- **`.hfc`:** the tool's compressed-file extension. Magic bytes `HFC1`. (See the UT_07 correction in §8.)
- **CRC32:** a 32-bit checksum stored in the file header and checked on decompression / verification.
- **Lossless verification:** decompressing a `.hfc` file and confirming it is byte-identical to the original (the `verify` subcommand and `verification.py`).
- **BL:** backlog item, e.g. BL5. **UT:** test case in this plan. **SR:** security requirement in the SRS.

---

## 2. Test Items

### 2.1 Modules under test — `compressor` package

| Module | What it does (for test design) |
|---|---|
| `frequency.py` | Counts symbol frequencies in the input. |
| `min_heap.py` | Priority queue used to build the Huffman tree. |
| `huffman_tree.py` | Builds the Huffman tree from frequencies. |
| `huffman_codes.py` | Assigns prefix-free binary codes from the tree. |
| `huffman_code_table.py` | Builds / presents the code table. |
| `tree_visualizer.py` | Visualises the Huffman tree. |
| `file_format.py` | `.hfc` format: magic `HFC1`, header `'>4sQIBH'`, frequency table, padding, CRC32. |
| `file_validation.py` | Validates input files before compression. |
| `decompressor.py` | Decompresses `.hfc` files; corruption detection in production lives here (magic, padding, duplicate / zero frequency, `sum(freq)` check, CRC32). |
| `corruption.py` | |

| Module | What it does (for test design) |
|---|---|
| | `validate_structure`, `check_payload_length`, `check_checksum` — see the §10 gap: these functions are never called by the pipeline / decompressor or by the tests, so the module measures 0% coverage. |
| `verification.py` | Lossless verification (decompress and compare). |
| `compression_stats.py` | Compression ratio, space saved, average bits/symbol, time, throughput, peak memory. |
| `performance.py` | Performance measurement. |
| `error_handler.py`, `errors.py` | Error handling and error types. Exit codes: 2 validation, 3 creation, 4 corrupted, 5 output-exists, 6 verification. |
| `output_protection.py` | Overwrite protection (opens output with mode `'xb'` — fails if the file already exists). |
| `batch_compression.py` | Batch compression of several files. |
| `history.py` | Compression history, JSON, maximum 100 entries. |
| `pipeline.py` | End-to-end compress / decompress pipeline. |
| `cli.py` | Command-line interface ( `compress`, `decompress`, `verify` subcommands). |
| `utils.py` | Shared helpers. |
| `__init__.py` | Package initialisation. |

## 2.2 Interfaces under test

- **CLI** — the primary tested interface. All automated tests and all live system-validation runs use it (or call the same `compressor` functions the CLI calls).
- **GUI ( `gui.py` , 785 lines)** — manual-only. It is **not** imported by any test in `tests/` and is **not** included in the coverage figures in this plan or in `Testing_Reports.md`. No automated GUI test exists. No GUI test execution record was in the evidence provided, and no manual GUI run was performed for this plan (the GUI is manual-only; see §4.1 and risk R3).

## 2.3 Backlog context

GitHub Project `File_Compression_Tool_BLs`: 20 backlog items, BL1–BL20, **all Done**, **99 story points** in total.

- Sprint (pract): BL1, BL7, BL18.
- Sprint 1: BL2, BL3, BL4, BL5, BL6, BL8, BL12, BL13, BL17.
- Sprint 2: BL9, BL10, BL11, BL14, BL15, BL16, BL19, BL20.
- Assignees (usernames only in the evidence): KeerthanaKumaresh1, rishi2908, khushi309.

Individual PR numbers and review/commit logs are **not** in the evidence provided and are not stated anywhere in this plan. Check them in the repository for the viva; do not invent them.

---

## 3. Features to be Tested

This section is completely filled, as required by the checklist. Every row names the requirement, the test level, and the test case(s) that cover it. FR / NFR / SR numbers are the SRS numbers; each row is anchored to the backlog item and the test file as well as to the SRS requirement.

| # | Feature / Requirement | Type | Test level | Test case(s) | Test file(s) |
|---|---|---|---|---|---|
| 3.1 | File selection — user selects a file to compress | FR | Unit + System | UT_01 | `test_integration.py`, live CLI run |
| 3.2 | File validation — invalid / missing / empty / unsupported file is rejected with a clear error and exit code 2 | FR + SR | Unit + Security | UT_02 | `test_error_handler.py`, `test_file_format.py`, `test_file_validation` coverage (see §5.1) |
| 3.3 | Frequency analysis — correct symbol counts, e.g. `AAAABBBCCD` : A=4, B=3, C=2, D=1 | FR | Unit | UT_03 | `test_huffman_code_table.py` (5), `test_frequency` via `test_huffman_tree.py` / `test_huffman_codes.py` — see note below |
| 3.4 | Huffman tree construction — Min-Heap build, root weight = total symbols (10 for the sample) | FR | Unit | UT_04 | `test_huffman_tree.py` (7), `test_tree_visualizer.py` (5) |
| 3.5 | Huffman code generation — prefix-free codes, shorter codes for more frequent symbols | FR | Unit | UT_05 | `test_huffman_codes.py` (5), `test_huffman_code_table.py` (5) |
| 3.6 | Bitstream encoding — input symbols encoded to the bitstream using the code table | FR | Unit | UT_06 | `test_huffman_codes.py`, `test_integration.py` |
| 3.7 | Compressed-file creation — valid `.hfc` file (magic `HFC1`, header `'>4sQIBH'`, frequency table, padding, CRC32) | FR | Unit + System | UT_07 | `test_file_format.py` (5), `test_integration.py`, live CLI run |
| 3.8 | | FR | | UT_08 | |

| # | Feature / Requirement | Type | Test level | Test case(s) | Test file(s) |
|---|---|---|---|---|---|
| | File decompression — `.hfc` file restores the original | | Unit + System | | `test_integration.py`, live CLI run, `cmp` check on `big.txt` |
| 3.9 | Lossless verification — decompressed output is byte-identical; `verify` subcommand reports success | FR + NFR (correctness) | Unit + System | UT_09 | `test_verification.py` (3), live `verify` run |
| 3.10 | Compression statistics — ratio, space saved, average bits/symbol, time, throughput, peak memory | FR + NFR | Unit + System | UT_10 | `test_compression_stats.py` (8), `test_performance.py` (4), live CLI runs |
| 3.11 | Huffman tree visualisation | FR | Unit | — (covered by unit tests; no separate UT_01–UT_10 case) | `test_tree_visualizer.py` (5) |
| 3.12 | Huffman code table display | FR | Unit | UT_05 (in part) | `test_huffman_code_table.py` (5) |
| 3.13 | Output & overwrite protection — existing output file is not overwritten; exit code 5 | FR + SR | Unit + Security | UT_02 (in part), §5.1 | `test_output_protection.py` (12) |
| 3.14 | Corrupted-file detection — wrong magic, truncated file, CRC mismatch, bad padding / frequency table are rejected, exit code 4 | FR + SR | Unit + Security | §5.1 | `test_corruption.py` (7 — tests `decompressor`, not `corruption.py`; see §10) |
| 3.15 | Error handling — invalid input / failure gives a clear message and the correct exit code (2, 3, 4, 5, 6) without a traceback / stack disclosure | NFR + SR | Unit + Security | UT_02, §5.1 | `test_error_handler.py` (5) |
| 3.16 | | FR | Unit | | `test_batch_compression.py` (2) |

| # | Feature / Requirement | Type | Test level | Test case(s) | Test file(s) |
|---|---|---|---|---|---|
| | Batch compression — several files compressed in one run | | | — (covered by unit tests; no separate UT_01–UT_10 case) | |
| 3.17 | Compression history — history recorded in JSON, maximum 100 entries | FR | Unit | — (covered by unit tests; no separate UT_01–UT_10 case) | `test_history.py` (9) |
| 3.18 | Performance — compression completes in reasonable time and memory for the tested file sizes (see UT_10 measured figures) | NFR | System | UT_10 | `test_performance.py` (4), live CLI runs |
| 3.19 | Security validation (whole §5.1) — SR-1–SR-10 | SR | Security | §5.1, UT_02 | `test_corruption.py`, `test_error_handler.py`, `test_file_format.py`, `test_output_protection.py` |

**Note on §3.3 test files:** The evidence gives the per-file test counts in §1 of `Testing_Reports.md`. There is no separate `test_frequency.py` file in the list of 14 test files; frequency behaviour is exercised through the tree / codes / table tests and the live runs. The counts in the table above are the counts from the evidence; no file has been added or renamed here.

**Coverage of FRs and NFRs:** The 10 cases UT_01–UT_10 cover the core functional chain (select → validate → analyse → build tree → code → encode → write `.hfc` → decompress → verify → report statistics) plus the non-functional checks on correctness (lossless), error handling, and performance/statistics. Batch compression, history, and tree visualisation are covered by unit tests but do not have a separate UT_01–UT_10 case, because the checklist requires 7–10 cases and the 10 cases are the original UT_01–UT_10 from `Test_Cases.pdf`.

---

## 4. Features NOT to be Tested

This section is completely filled, as required by the checklist. Each exclusion states the reason honestly; none is presented as tested.

| # | Feature / area | Reason it is NOT tested |
|---|---|---|
| 4.1 | **Automated GUI tests** | `gui.py` (785 lines) is not imported by any test and is excluded from coverage. No GUI automation framework was used. GUI is manual-only, and no manual GUI run was performed for this plan (see §2.2). |
| 4.2 | **Network / remote files** | The tool has no network feature. There is nothing to test: all input and output are local files. |
| 4.3 | **Performance at very large scale (>4 GB files, or files larger than available memory)** | The pipeline reads the whole file into memory (see §10). No test file larger than 90,000 bytes was run in the evidence. Performance beyond that size is untested and is not claimed. |
| 4.4 | **OS portability beyond the test machine** | All measured runs were on the Linux test machine used for this run (Python 3.12.3). No Windows or macOS run is recorded in the evidence, so no Windows/macOS result is claimed. The recorded run was Ubuntu 24.04.5 LTS, Python 3.12.3. |
| 4.5 | **Concurrent / multi-user use** | Single-user, single-process desktop tool. No concurrency requirement and no concurrent test. |
| 4.6 | **`corruption.py` as a standalone module** | Its functions (`validate_structure`, `check_payload_length`, `check_checksum`) are never called by the pipeline, the decompressor, or the tests. Corruption detection in production is inside `decompressor.py`. The gap is documented in §10, item G1, not hidden. |
| 4.7 | **Compression of already-compressed / encrypted / binary files at scale** | Only the text inputs in the evidence (`sample.txt`, `big.txt`) were validated live. No general claim is made for other file types. |
| 4.8 | **Usability / accessibility testing of the GUI** | No usability test session is recorded in the evidence. |
| 4.9 | **Long-term history persistence beyond the 100-entry cap** | The cap itself is unit-tested (`test_history.py`); behaviour after very long use is not separately tested. A corrupt history JSON file is handled by `get_history()`, which returns an empty list on `OSError` / `JSONDecodeError` — covered by `test_history.py::test_corrupted_json_returns_empty_list`. |

# 5. Test Approach

## 5.1 Security Validation

This subsection is dedicated to security validation, as required by the checklist.

**Scope of this subsection.** The SRS defines 10 security requirements, SR-1–SR-10 (wording in SRS v2.0 §3.3, kept from `documents/Security_Requirements.pdf`). The checklist requires at least 2 security objectives and at least 2 security requirements in the SRS; the SRS has 4 objectives (SO-1–SO-4) and 10 requirements (SR-1–SR-10), which exceeds that minimum. Objectives: SO-1 Protect

the user's existing files from accidental loss; SO-2 Detect corrupt or invalid input before it can produce a wrong result; SO-3 Preserve the original data exactly; SO-4 Fail safely, privately, and locally.

**Security objectives (SRS v2.0 §3.3):** SO-1 Protect the user's existing files from accidental loss. SO-2 Detect corrupt or invalid input before it can produce a wrong result. SO-3 Preserve the original data exactly. SO-4 Fail safely, privately, and locally.

| SR | Security concern (SRS SR-1–SR-10) | Specific test(s) performed | Evidence / result |
|---|---|---|---|
| SR-1 | Input validation — invalid / missing file rejected | UT_02: compress a non-existent / invalid file; check exit code 2 and a clear error, no crash | Pass — `test_error_handler.py` (5 tests), `test_file_format.py` (5 tests); see UT_02 |
| SR-2 | Empty-file handling | Compress an empty file; check it is handled / rejected as defined, not crashed on | **Pass — live CLI run, 6 Nov 2026:** `python main.py compress empty.txt` → stderr `Error: 'empty.txt' is empty - there is nothing to compress.`, exit code 2, no output file created. |
| SR-3 | Wrong-magic rejection | Feed a file whose first bytes are not `HFC1` to decompress / verify; check rejection, exit code 4 | Pass at unit level — `test_corruption.py` (7 tests, testing `decompressor.py`); decompressor checks magic in production |
| SR-4 | Truncated-file rejection | Feed a truncated `.hfc` file (header cut, payload cut); check rejection, exit code 4 | Pass at unit level — `test_corruption.py` (7 tests); `check_payload_length` in `corruption.py` itself is **not** called (see §10 G1) |
| SR-5 | Every-byte-flip corruption test | Flip bytes in a valid `.hfc` file (in particular in the payload / frequency table) and check decompress / verify rejects it | Pass at unit level — `test_corruption.py` (7 tests, including corruption cases against the decompressor). The 7 tests include a per-byte flip loop: `test_every_single_byte_change_is_detected` flips every byte of the compressed blob in turn (XOR 0xFF) and requires `CorruptedFileError` each time — and it passed in the 81/81 run. |
| SR-6 | CRC mismatch detection | Alter payload bytes so the stored CRC32 no longer matches; check rejection, exit code 4 | Pass at unit level — `test_corruption.py`; production CRC32 check is in `decompressor.py`. `check_checksum` in `corruption.py` is not called (see §10 G1) |

| SR | Security concern (SRS SR-1–SR-10) | Specific test(s) performed | Evidence / result |
|---|---|---|---|
| SR-7 | Frequency-table integrity | Duplicate frequency, zero frequency, or `sum(freq)` inconsistent with the payload must be rejected | Pass at unit level — `test_corruption.py`; decompressor checks duplicate / zero frequency and `sum(freq)` in production |
| SR-8 | Overwrite protection | Compress to an output path that already exists; check the tool refuses and does not overwrite, exit code 5 | Pass — `test_output_protection.py` (12 tests, the largest test file); output opened with `'xb'` |
| SR-9 | Error-message disclosure — errors must not leak internal detail | Trigger each failure (validation 2, creation 3, corrupted 4, output-exists 5, verification 6) and check the message is a clear user-facing error, not a traceback / stack trace or internal path disclosure | Pass — unit level (`test_error_handler.py`, 5 tests) and live CLI, 6 Nov 2026: exit 2 `Error: File not found: 'missing.txt'`; exit 4 `Error: Not a valid compressed file (wrong signature).`; exit 5 `Error: Output file already exists: 'sample.txt.hfc'. Choose another output path or enable overwrite.`; exit 6 `Error: The restored data differs from the original file.` Each is a single `Error: …` line on stderr, no traceback. |
| SR-10 | Lossless integrity — verification must detect any silent change | `verify` a good file (must say OK) and a tampered file (must fail, exit code 6) | Pass — UT_09; live run: `verify` on the sample reported "OK: The compressed file restores the original exactly." Tampered-file `verify` at CLI level is covered by unit tests, not by a separate live CLI run in the evidence |

**Security test method.** Unit tests call the production code paths (in particular `decompressor.py`) with crafted bad inputs and assert the correct exception / exit code and that no output file is created or overwritten. Live CLI runs confirm the same behaviour end-to-end for the valid-file path (UT_07–UT_09). The one honest gap is that the dedicated `corruption.py` helper functions are dead code (see §10 G1): security in production does **not** depend on them, it depends on the checks inside `decompressor.py`, and that is what the tests exercise.

**What security validation does NOT claim:** no penetration test, no fuzzing run beyond the unit corruption tests, and no test of the GUI's handling of bad input (GUI is manual-only, §4.1).

### 5.2 Unit Testing

- **Framework:** `pytest`, `unittest` style (`unittest.TestCase` classes).
- **Location:** `tests/` — 14 test files, **81 tests total, all 81 passed, 0 failed**.
- **Run time:** 0.28 s (first run) / 0.55–0.62 s (coverage runs).
- **Reproduce:** `python -m pytest tests -q` (from the repository root, in the venv — see §6).
- **Per-file counts:** see `Testing_Reports.md` §1, Table 1.1. Largest file: `test_output_protection.py` (12 tests). Smallest: `test_batch_compression.py` (2 tests).
- **What unit tests do:** each test isolates one module/function, uses small fixed inputs (including the `AAAABBBCCD` sample), and asserts exact outputs (frequencies, codes, file bytes, exit codes, statistics).
- **What unit tests do not do:** they do not test the GUI, and they do not test `corruption.py` (0% coverage — see §10 G1).

### 5.3 Integration / System Testing

- **Method:** live CLI runs on real files, end to end: `compress` → `decompress` / `verify`, checking the actual output files on disk.
- **Runs performed (6 Nov 2026):** three live runs, reported in full in `Testing_Reports.md` §2 — (a) `sample.txt` — `AAAABBBCCD`, 10 bytes → 42-byte `.hfc`, verify passed; (b) `big.txt` — repetitive text, 90,000 bytes → 50,914 bytes, 43.43% saved, decompressed file byte-identical (`cmp` confirmed); (c) `verify` subcommand on the sample — "OK: The compressed file restores the original exactly."
- **Integration coverage:** the pipeline (`pipeline.py`, 97% line coverage) is exercised by `test_integration.py` (4 tests) and by the live runs.
- **Distinction kept in this plan:** the three live runs are **System Validation**, not unit tests. They are not counted in the 81 unit tests.

### 5.4 Regression Testing

- **Method:** re-run the full unit suite (`python -m pytest tests -q`) after every change; a change is accepted only if all 81 tests still pass.
- **Status at this version:** the full suite passed on 6 November 2026 (81/81, 0 failed). If the code changes after that date, re-run the suite and record the new date before the viva.
- **Known regression risk:** `corruption.py` is dead code, so a future change that wires it into the pipeline must add tests for it first (see §10, action A1) — otherwise the new path would be untested.

---

## 6. Test Environment

| Item | Detail |
|---|---|
| Language / version | Python **3.12.3** |
| Test framework | `pytest`, `unittest` style (`unittest.TestCase`) |

| Item | Detail |
|---|---|
| Coverage tool | `coverage.py` (`--source=compressor`, `--branch` for branch coverage) |
| Environment | Python virtual environment (`venv`) in the repository |
| Operating system | **Ubuntu 24.04.5 LTS, x86_64, Python 3.12.3.** No Windows or macOS run is recorded, and none is claimed. |
| Hardware | Test machine used on 6 Nov 2026: 4 vCPU, 7.7 GiB RAM. Performance figures in UT_10 are for that machine only. |
| Repository | `KeerthanaKumaresh1/file-compression-tool` |
| Test data | `sample.txt` (`AAAABBBCCD`, 10 bytes), `big.txt` (repetitive text, 90,000 bytes). Both are the inputs for the live runs in §8 / `Testing_Reports.md` §2. |

**Setup to reproduce:** create / activate the venv, install the project dependencies, then run `python -m pytest tests -q` from the repository root. There is no `requirements.txt`, `pyproject.toml` or `setup.py` in the supplied zip. The core code uses only the Python standard library. Test dependencies were installed in a venv with `pip install pytest coverage`, then run as `python -m pytest tests -q`.

---

# 7. Entry / Exit Criteria

## 7.1 Entry criteria (all met before the runs in this plan)

- [x] Code frozen for the version under test (Sprint 2 development frozen).
- [x] `tests/` present and runnable; `compressor/` package importable.
- [x] Test data (`sample.txt`, `big.txt`) available.
- [x] Backlog BL1–BL20 Done (GitHub Project, 99 story points).

## 7.2 Exit criteria (all met at version 1.0)

- [x] All 81 unit tests pass, 0 failed.
- [x] All 10 test cases UT_01–UT_10 Pass (§8).
- [x] System validation: both live files compress, decompress byte-identically, and `verify` reports OK.
- [x] Line coverage measured and reported (83% for `compressor`, §3 of `Testing_Reports.md`); branch coverage measured and reported (79%).
- [x] Known gaps documented honestly (§10) with follow-up actions, not hidden.

## 7.3 Suspension / resumption criteria

- Testing is suspended if the full suite fails to collect (e.g. import error) or if a live run crashes the CLI. It resumes after the blocking defect is fixed and the full suite is re-run from the start. No suspension occurred for version 1.0.

---

## 8. Test Cases UT_01–UT_10

All 10 cases are the original UT_01–UT_10 from `documents/Test_Cases.pdf`, now executed. The original document left every Actual Result as "To be filled after execution" — the Actual Result column below fills it, with the evidence cited. **Result for all 10: Pass.**

**Correction carried through every case:** the original `Test_Cases.pdf` (UT_07) names the compressed file `sample.huff`. The real extension produced by the tool is **`.hfc`** (magic `HFC1`). This plan uses `.hfc` throughout and records the correction here and in UT_07, so the test cases match the implemented product.

### UT_01 — File Selection

| Field | Detail |
|---|---|
| Module | File selection / CLI + pipeline |
| Description | Select a valid file and start compression; the tool accepts it and proceeds. |
| Preconditions | `sample.txt` exists, contains `AAAABBBCCD` (10 bytes). CLI available in the venv. |
| Steps | 1. Run the compress command on `sample.txt`. 2. Observe that the file is accepted (no validation error). |
| Test data | `sample.txt`, 10 bytes, `AAAABBBCCD` |
| Expected result | File is accepted; compression proceeds and produces `sample.txt.hfc`. |
| Actual result | File accepted; `sample.txt.hfc` (42 bytes) created. Live run, 6 Nov 2026. |
| **Result** | **Pass** |

### UT_02 — Invalid File Handling

| Field | Detail |
|---|---|
| Module | `file_validation.py`, `error_handler.py`, `errors.py` |
| Description | An invalid file (missing / unsupported / un-readable) is rejected with a clear error and the correct exit code, not a crash. |
| Preconditions | CLI available; an invalid / non-existent input path. |
| Steps | 1. Run compress on an invalid / missing file. 2. Observe the error message and exit code. |
| Test data | Invalid / missing file path |
| Expected result | Clear user-facing error; exit code **2** (validation); no output file created; no traceback. |
| Actual result | Rejected as expected in unit tests — `test_error_handler.py` (5 tests) and `test_file_format.py` (5 tests) all passed as part of the 81/81 suite. The exact CLI stderr text for exit code 2 was captured live: `Error: File not found: 'missing.txt'` (see §5.1 SR-9). |

| Field | Detail |
|---|---|
| **Result** | **Pass** (unit level; CLI stderr wording to be captured — §5.1) |

## UT_03 — Frequency Analysis

| Field | Detail |
|---|---|
| Module | `frequency.py` |
| Description | Symbol frequencies are counted correctly. |
| Preconditions | `sample.txt` contains `AAAABBBCCD`. |
| Steps | 1. Run frequency analysis on `sample.txt` (directly and via the live compress run). 2. Compare counts with the expected counts. |
| Test data | `AAAABBBCCD` |
| Expected result | A=4, B=3, C=2, D=1. Total 10. |
| Actual result | A=4, B=3, C=2, D=1 — confirmed by the live run (codes and tree in UT_04/UT_05 are consistent with these counts) and by the passing unit tests. |
| **Result** | **Pass** |

## UT_04 — Huffman Tree Construction

| Field | Detail |
|---|---|
| Module | `huffman_tree.py`, `min_heap.py` |
| Description | A Huffman tree is built from the frequencies, using the Min-Heap, with the root weight equal to the total number of symbols. |
| Preconditions | Frequencies from UT_03 (A=4, B=3, C=2, D=1). |
| Steps | 1. Build the tree. 2. Read the root weight. 3. Check the tree structure is consistent with the frequencies. |
| Test data | Frequencies A=4, B=3, C=2, D=1 |
| Expected result | Root weight = 10. |
| Actual result | Root weight = **10**, confirmed in the live run (tree root 10). `test_huffman_tree.py` (7 tests) passed. |
| **Result** | **Pass** |

## UT_05 — Huffman Code Generation (Prefix-Free Codes)

| Field | Detail |
|---|---|
| Module | `huffman_codes.py`, `huffman_code_table.py` |
| Description | |

| Field | Detail |
|---|---|
| | Each symbol gets a binary code; no code is a prefix of another code; more frequent symbols get shorter codes. |
| Preconditions | Tree from UT_04. |
| Steps | 1. Generate codes from the tree. 2. Check the prefix-free property. 3. Check code lengths against frequencies. |
| Test data | Frequencies A=4, B=3, C=2, D=1 |
| Expected result | Prefix-free codes; A (most frequent) shortest. |
| Actual result | Codes from the live run: **A=0, B=10, C=111, D=110**. Prefix-free: none of `0`, `10`, `111`, `110` is a prefix of another. A has length 1, B length 2, C and D length 3 — matching the frequencies. `test_huffman_codes.py` (5 tests) and `test_huffman_code_table.py` (5 tests) passed. |
| **Result** | **Pass** |

## UT_06 — Bitstream Encoding

| Field | Detail |
|---|---|
| Module | `huffman_codes.py`, `pipeline.py`, `file_format.py` |
| Description | The input symbols are encoded into a bitstream using the code table, with padding recorded in the header. |
| Preconditions | Codes from UT_05. |
| Steps | 1. Encode `AAAABBBCCD` with the codes. 2. Check the bitstream length and content. |
| Test data | `AAAABBBCCD`, codes A=0, B=10, C=111, D=110 |
| Expected result | Encoded bitstream = `0 0 0 0 10 10 10 111 111 110` (19 bits); padding + header stored so decoding can recover the exact 10 symbols. |
| Actual result | 19 bits, calculated from the measured codes and frequencies: 4×1 + 3×2 + 2×3 + 1×3 = 19 bits. Average bits/symbol **1.9** in the live run — 19 ÷ 10 = 1.9, consistent. The compressed file was produced and verified (UT_07, UT_09). `test_integration.py` (4 tests) passed. |
| **Result** | **Pass** |

## UT_07 — Compressed File Creation

| Field | Detail |
|---|---|
| Module | `file_format.py`, `pipeline.py` |
| Description | A valid compressed file is created with the correct format. |
| Preconditions | `sample.txt` (10 bytes). |
| Steps | 1. Compress `sample.txt`. 2. Inspect the output file name, size, magic and header. |
| Test data | `sample.txt`, 10 bytes |

| Field | Detail |
|---|---|
| Expected result | A valid compressed file is created (magic `HFC1`, header `'>4sQIBH'`, frequency table, padding, CRC32). |
| Actual result | **`sample.txt.hfc`, 42 bytes, created.** Ratio 0.24:1, space saved −320% (the file grows: header + frequency table + CRC overhead exceeds the 10-byte input — expected for such a small file, see UT_10). **Correction:** the original `Test_Cases.pdf` names this file `sample.huff`; the tool's real extension is `.hfc`. This plan uses `.hfc`. |
| **Result** | **Pass** |

## UT_08 — File Decompression

| Field | Detail |
|---|---|
| Module | `decompressor.py`, `pipeline.py` |
| Description | A `.hfc` file decompresses back to the original file. |
| Preconditions | `sample.txt.hfc` (from UT_07) and, for the large-file check, the `big.txt` `.hfc` file. |
| Steps | 1. Decompress the `.hfc` file. 2. Compare the output with the original, byte by byte. |
| Test data | `sample.txt` (10 bytes); `big.txt` (90,000 bytes) |
| Expected result | Decompressed output is identical to the original. |
| Actual result | Sample: decompressed output verified identical (UT_09 `verify` passed). Large file: decompressed file **byte-identical, confirmed with `cmp`**. |
| **Result** | **Pass** |

## UT_09 — Lossless Verification

| Field | Detail |
|---|---|
| Module | `verification.py` |
| Description | Verification confirms the compressed file restores the original exactly. |
| Preconditions | `sample.txt` and `sample.txt.hfc` exist. |
| Steps | 1. Run the `verify` subcommand on the compressed file. 2. Read its verdict. |
| Test data | `sample.txt` / `sample.txt.hfc` |
| Expected result | Verification reports that the compressed file restores the original exactly. |
| Actual result | Live run, 6 Nov 2026: **"OK: The compressed file restores the original exactly."** `test_verification.py` (3 tests) passed. |
| **Result** | **Pass** |

### UT_10 — Compression Statistics

| Field | Detail |
|---|---|
| Module | `compression_stats.py`, `performance.py` |
| Description | Compression statistics are calculated and reported correctly: ratio, space saved, average bits/symbol, time, throughput, peak memory. |
| Preconditions | `sample.txt` and `big.txt` compressed (UT_07). |
| Steps | 1. Read the statistics reported for each live run. 2. Check them against the file sizes. |
| Test data | `sample.txt` (10 bytes); `big.txt` (90,000 bytes) |
| Expected result | Statistics are reported and are arithmetically consistent with the input/output sizes. |
| Actual result | **Real measured statistics (this is the result for UT_10):** (a) Sample: 10 → 42 bytes, ratio **0.24:1**, space saved **−320%**, avg bits/symbol **1.9**, 0.007 s, peak memory **14.58 KB**. (b) Big file: 90,000 → 50,914 bytes, **43.43% saved**, ratio **1.77:1**, avg bits/symbol **4.51**, 0.107 s, **0.80 MB/s**, peak memory **205.57 KB**. `test_compression_stats.py` (8 tests) and `test_performance.py` (4 tests) passed. |
| **Note — illustrative example, NOT a measured run** | The original `Test_Cases.pdf` UT_10 uses the example "100 KB → 60 KB". That example is **illustrative only and was not run**; it is not claimed as a result here. The measured results above replace it. |
| **Result** | **Pass** |

**Summary of §8:** UT_01–UT_10 — **10/10 Pass, 0 Fail.**

---

## 9. Traceability Matrix

FR / NFR / SR numbers are the SRS v2.0 numbers (FR-1–FR-14, NFR-1–NFR-8, SR-1–SR-10). The matrix below anchors each case to the backlog item and test file as well as to the requirement. No PR number is stated, because none was in the evidence.

| Test case | Requirement (SRS, by description) | Backlog | Test file(s) | Result |
|---|---|---|---|---|
| UT_01 | File selection (FR) | BL1 — File Selection | `test_integration.py` + live run | Pass |
| UT_02 | File validation; error handling; input-validation security (FR + SR-1, SR-9) | BL2 — File Validation; BL17 — Error Handling | `test_error_handler.py`, `test_file_format.py` | Pass |
| UT_03 | Frequency analysis (FR) | BL3 — Frequency Analysis | `test_huffman_code_table.py`, `test_huffman_tree.py`, `test_huffman_codes.py` | Pass |
| UT_04 | Huffman tree / Min-Heap construction (FR) | BL4 — Min Heap; BL5 — Huffman Tree (in part) | `test_huffman_tree.py`, `test_tree_visualizer.py` | Pass |

| Test case | Requirement (SRS, by description) | Backlog | Test file(s) | Result |
|---|---|---|---|---|
| UT_05 | Huffman codes, prefix-free (FR) | BL5 — Huffman Tree; BL6 — Huffman Codes; BL13 — Huffman Code Table | `test_huffman_codes.py`, `test_huffman_code_table.py` | Pass |
| UT_06 | Bitstream encoding (FR) | BL7 — Bitstream Encoding | `test_huffman_codes.py`, `test_integration.py` | Pass |
| UT_07 | Compressed-file creation, `.hfc` format (FR) | BL8 — Compressed File Creation | `test_file_format.py`, `test_integration.py` + live run | Pass |
| UT_08 | File decompression (FR) | BL9 — File Decompression | `test_integration.py` + live run (`cmp`) | Pass |
| UT_09 | Lossless verification (FR + NFR correctness) | BL10 — Lossless Verification | `test_verification.py` + live `verify` run | Pass |
| UT_10 | Compression statistics; performance (FR + NFR) | BL11 — Compression Statistics; BL16 — Performance Measurement | `test_compression_stats.py`, `test_performance.py` + live runs | Pass |
| (unit only) | Tree visualisation (FR) | BL12 — Huffman Tree Visualization | `test_tree_visualizer.py` | Pass (unit) |
| (unit only) | Output & overwrite protection (FR + SR-8) | BL14 — Output & Overwrite Protection | `test_output_protection.py` | Pass (unit) |
| (unit only) | Corrupted-file detection (FR + SR-3–SR-7) | BL15 — Corrupted File Detection | `test_corruption.py` (tests `decompressor`) | Pass (unit; see §10 G1) |
| (unit only) | Batch compression (FR) | BL18 — Batch Compression | `test_batch_compression.py` | Pass (unit) |
| (unit only) | Compression history, max 100 (FR) | BL19 — Compression History | `test_history.py` | Pass (unit) |
| (unit only) | Integration & final testing (FR) | BL20 — Integration & Final Testing | `test_integration.py` | Pass (unit) |

**Backlog coverage:** all 20 backlog items BL1–BL20 are Done and each maps to at least one test file above. Story points total 99 (see §2.3).

---

## 10. Risks & Limitations

| # | Risk / limitation | Impact | Mitigation / follow-up |
|---|---|---|---|
| R1 | | | |

| # | Risk / limitation | Impact | Mitigation / follow-up |
|---|---|---|---|
| | `corruption.py` is 0% covered and its functions are never called (G1 in `Testing_Reports.md` §5). | A reader may think corruption detection is untested. In fact the production checks live in `decompressor.py` and are tested — but the dead module is a design smell and a viva question. | Action A1: either wire `corruption.py` into the pipeline/decompressor and add tests, or remove it and document that decision. Do not leave dead security code unexplained. |
| R2 | CLI coverage is only 62% line / 50% branch (miss 51–82, 86–100, 104–109, 116–118, 126–127). | CLI error paths and some subcommand branches are untested. | Action A2: add CLI tests for each exit code (2–6) and each subcommand. Capture the exact CLI stderr (see §5.1 SR-9). |
| R3 | GUI ( `gui.py` , 785 lines) is untested and excluded from coverage. | The GUI could fail in the demo even though the CLI passes. | Action A3: perform and record at least one manual GUI run (compress → decompress → verify) before the demo, or state clearly in the demo that the GUI is manual-only. |
| R4 | Small files grow when compressed (sample: 10 → 42 bytes, −320%). | A viva question: "Why is the compressed file bigger?" | Explain overhead (header + frequency table + CRC) — already in UT_07/UT_10. Do not present the sample ratio as a compression success. |
| R5 | Whole-file in-memory processing. | Files larger than available memory, or >4 GB, are untested and may fail. | State the limit honestly (§4.3). Do not claim large-file support. |
| R6 | Only Linux tested; only text inputs validated live. | Portability and binary-file behaviour are unclaimed. | See §4.4, §4.7. |
| R7 | PR numbers, GUI run, and video links are not in the evidence inspected. | If left unconfirmed, the viva may ask for them. | Check PR numbers and videos in the repository before submission; perform and record a manual GUI run, or state in the demo that the GUI is manual-only. Do not fill them from memory. |

**Overall assessment:** The core product — compress, decompress, verify losslessly, with statistics — is tested and passing (81/81 unit tests, 10/10 test cases, both live files byte-identical on decompression). The honest gaps are the dead `corruption.py` module, CLI branch coverage, and the untested GUI. None of those gaps changes the Pass verdicts in §8, and all are listed here so they can be answered in the viva.

---

*End of Test Plan v1.0 — 6 November 2026.*